Practical No. 4

Shell Program to Perform Mathematical Operations

1. AIM

To write a Shell script that accepts two numbers from the user and performs Addition, Subtraction, Multiplication, Division and Modulus, and displays the results on the screen.

2. REQUIREMENTS

- Operating System: Ubuntu / any Linux or Unix system
- Shell: Bash (Bourne Again Shell)
- Text editor: Nano / Vim / Gedit / VS Code
- Terminal for running the script
- Hardware: computer, keyboard (input), monitor (output)

3. SHORT THEORY

Shell script: a file containing a list of commands that the shell runs one after another. It is used for automation and routine tasks.

- Shebang (#!/bin/bash): first line, tells the system to run the script using Bash.
- echo: prints messages and results on the screen.
- read: takes input from the user and stores it in a variable.
- Variable: a named place to store a value, e.g. num1, num2.
- Arithmetic expansion $(( )): Bash calculates the expression inside and returns the result (integers only).
- Operators: + (add), - (subtract), * (multiply), / (divide), % (modulus = remainder).
- if statement: checks a condition. Used here to avoid division by zero.

4. CODE (file name: math_operations.sh)

```
#!/bin/bash
clear
echo "------ Mathematical Operations in Shell Script ------"

read -p "Enter First Number: " num1
read -p "Enter Second Number: " num2

echo ""
echo "------ Results ------"

add=$((num1 + num2))
echo "Addition ($num1 + $num2) = $add"

sub=$((num1 - num2))
echo "Subtraction ($num1 - $num2) = $sub"

mul=$((num1 * num2))
echo "Multiplication ($num1 * $num2) = $mul"

if [ $num2 -ne 0 ]; then
  div=$((num1 / num2))
  echo "Division ($num1 / $num2) = $div"
  mod=$((num1 % num2))
  echo "Modulus ($num1 % $num2) = $mod"
```

```
else
  echo "Division = Not Possible (Division by Zero)"
  echo "Modulus = Not Possible (Division by Zero)"
fi
```

5. HOW TO EXECUTE (step by step)

Command / Key	What it does
`Ctrl + Alt + T`	Open the Ubuntu terminal.
`cd ~`	Go to the home folder.
`nano math_operations.sh`	Create and open the script file in the nano editor.
`(paste code)`	Paste the code using Ctrl + Shift + V.
`Ctrl+O, Enter, Ctrl+X`	Save the file, then exit nano.
`chmod +x math_operations.sh`	Give the file execute permission.
`./math_operations.sh`	Run the script. Enter two numbers when asked.

Sample run: First number = 10, Second number = 3 -> Addition = 13, Subtraction = 7, Multiplication = 30, Division = 3, Modulus = 1.

6. LINE-BY-LINE EXPLANATION

Code	Meaning
`#!/bin/bash`	Shebang. Tells Linux to run this file with the Bash shell.
`clear`	Clears the terminal screen so the output looks clean. Not compulsory.
`echo "------ ... ------"`	Prints the heading of the program.
`read -p "Enter First Number: " num1`	read takes input. -p shows the prompt text. The typed value is stored in variable num1.
`read -p "Enter Second Number: " num2`	Same as above. The second value is stored in num2.
`echo ""`	Prints a blank line for spacing.
`echo "------ Results ------"`	Prints the results heading.
`add=$((num1 + num2))`	Adds the two numbers using $(( )) and stores the answer in variable add. No spaces around =.
`echo "Addition ($num1 + $num2) = $add"`	Prints the values and the result. $ before a name gives the value stored in that variable.
`sub=$((num1 - num2))`	Subtracts num2 from num1 and stores it in sub.
`mul=$((num1 * num2))`	Multiplies the two numbers and stores it in mul.
`if [ $num2 -ne 0 ]; then`	Checks that num2 is NOT equal to zero (-ne). Needed because we cannot divide by zero.
`div=$((num1 / num2))`	Integer division. Gives the quotient only (10/3 = 3).
`mod=$((num1 % num2))`	Modulus. Gives the remainder after division (10%3 = 1).
`else`	Runs when the condition is false, i.e. num2 is zero.
`echo "... Not Possible ..."`	Prints an error message instead of crashing.

Code	Meaning
`fi`	Ends the if block (if spelled backwards).

7. MEANING OF SYMBOLS

Symbol	Use
`#`	Comment (ignored by Bash). With ! on the first line it forms the shebang #!.
`$`	Gets the value of a variable. $num1 = the value stored in num1.
`=`	Assigns a value. No spaces allowed on either side: add=5.
`$(( ))`	Arithmetic expansion. Does the calculation inside and gives the result.
`[ ]`	Test brackets for a condition. A space is needed after [and before].
`-ne`	Not equal. Others: -eq equal, -gt greater, -lt less.
`;`	Separates two commands on one line (here the test and then).
`" "`	Double quotes. Text inside is printed and variables inside are replaced by their values.
`./`	Means 'in the current folder'. Used to run the script.
`+x (chmod)`	Adds the execute permission to the file.

8. CONCLUSION

Thus, we have successfully written and executed a Shell script that takes two numbers as input and performs addition, subtraction, multiplication, division and modulus. We learned to use read, echo, variables, arithmetic expansion $(( )) and the if statement to handle division by zero.

9. QUICK VIVA ANSWERS

- expr vs $(( )): expr is an external command with awkward syntax; $(( )) is built into Bash, faster and simpler.
- Decimal division: use bc, e.g. echo "scale=2; 10/3" | bc gives 3.33.
- sh vs bash: sh is a basic POSIX shell; bash has extra features. Bash is better.
- Command substitution: stores a command's output in a variable, e.g. result=$(echo "10/3" | bc -l).
- echo vs printf: printf gives better formatting control, e.g. printf "%.2f\n" 3.333.
- Many inputs in one read: read a b (then type: 5 7).
- Why chmod +x: without execute permission, Linux will not run the file as a program.